

The Slip Model, From First Principles

Machine learning and neural networks, applied
to slip-adaptive state estimation in go2_eskf

Package: go2_eskf | ROS 2 Jazzy | PyTorch + Eigen

A self-contained tutorial: no prior ML background assumed.

0. What this document is

The go2_eskf localization filter uses a small neural network to decide, moment to moment, how much to trust the robot's leg odometry. This document explains that network from the ground up. It assumes no machine learning background. We start with what 'learning from data' even means, build up to neural networks, show exactly how the network is written and trained in PyTorch, and finish with how it is deployed inside the estimator with no machine-learning runtime on the robot.

Reading path:

- Section 1 - the problem in robotics terms (why we need this at all).
- Section 2 - machine learning fundamentals (models, data, loss, training).
- Section 3 - neural network fundamentals (neurons, layers, activations).
- Section 4 - how PyTorch builds and trains the network in this repo.
- Section 5 - the slip model specifically (features, architecture, output).
- Section 6 - deployment: training in Python, inference in C++.
- Section 7 - run it yourself. Section 8 - limits. Appendix - glossary.

1. The problem, in robotics terms

The Unitree Go2 is a legged robot. To know where it is, the filter fuses an IMU (inertial sensor), GPS, and 'leg odometry'. Leg odometry estimates the body's velocity from the leg joint angles, using one key assumption: a foot that is planted on the ground does not move in the world. If you know a foot is stationary and you know the leg geometry, the motion of the joints tells you how the body moved relative to that foot.

That assumption fails the instant a foot SLIPS. The joints still report motion, so leg odometry confidently reports a body velocity - but it is wrong. A standard estimator that always trusts leg odometry by a fixed amount will absorb that wrong measurement and corrupt its position estimate.

The fix we want: detect slip from sensor signals as it happens, and during slip, tell the estimator to trust leg odometry LESS and lean on the IMU and GPS instead. 'Detect slip from signals' is a pattern-recognition task, and that is exactly what machine learning is good at. The rest of this document is how we turn that idea into working code.

2. Machine learning fundamentals

Machine learning is a way to build a function when you cannot easily write the rules by hand, but you DO have examples of inputs paired with the right outputs. Instead of programming the rule, you choose a flexible function with adjustable knobs (parameters) and let an algorithm tune the knobs until the function reproduces your examples. Then you hope it also works on new inputs it never saw.

2.1 Models, parameters, and prediction

A 'model' is just a parameterized function $y = f(x; \theta)$. Here x is the input (a list of numbers called FEATURES), y is the output (the PREDICTION), and θ is the set of internal numbers (the PARAMETERS, also called weights) that the learning algorithm will adjust. Different values of θ give different functions. Learning means searching for the θ that makes f behave the way the examples say it should.

2.2 Supervised learning: features and labels

We use SUPERVISED learning: every training example is a pair (x, y_{true}) , an input and the correct answer (the LABEL). The input x is a feature vector - a fixed-length list of numbers describing the situation. For the slip model, x has 8 numbers (commanded-vs-measured velocity disagreement, joint speeds, contact fraction, and so on - Section 5.2). The label y_{true} is 1.0 if that situation was slipping and 0.0 if it was not.

A DATASET is a big table: each row is one example, the first columns are the features, the last column is the label. Training consumes this table.

2.3 Classification vs regression

If the output is a category (slip / no-slip), the task is CLASSIFICATION. If it is a continuous quantity (e.g. predict a temperature), it is REGRESSION. The slip model is BINARY classification: two classes. We do not just want a hard yes/no though - we want a confidence between 0 and 1, a 'slip score' s , which we will later turn into a smooth dial on how much to trust leg odometry. A score near 1 means 'almost certainly slipping'.

2.4 The loss function: measuring how wrong we are

To tune the parameters we need a single number that says how badly the model is doing on the training data. That number is the LOSS. Lower is better; training means making the loss small. For binary classification with a probability output p in $[0,1]$ and true label y in $\{0,1\}$, the standard loss is BINARY CROSS-ENTROPY (BCE):

```
loss for one example:
  L = -[ y * log(p) + (1 - y) * log(1 - p) ]

  if y = 1 and p -> 1 : L -> 0      (confident and correct)
  if y = 1 and p -> 0 : L -> +inf   (confident and WRONG)

total loss = average of L over all training examples
```

BCE punishes confident wrong answers very hard and rewards confident right ones. Minimizing it pushes the predicted probabilities toward the labels.

2.5 Training is just minimizing the loss

The loss depends on the parameters θ (through the prediction p). So 'find good parameters' becomes 'find the θ that minimizes the loss' - an optimization problem. The workhorse method is GRADIENT DESCENT.

The GRADIENT of the loss with respect to a parameter tells you which way to nudge that parameter to make the loss go UP. So we step the opposite way, downhill, by a small amount called the LEARNING RATE (lr):

```
for each parameter w:
  w <- w - lr * (dLoss / dw)

repeat for many passes over the data (each pass is an EPOCH)
```

Each step lowers the loss a little. After many steps the parameters settle near a minimum and the model fits the data. The only real subtlety is computing $d\text{Loss}/dw$ for every parameter efficiently - neural network libraries do this automatically (Section 4.1).

2.6 Generalization and overfitting

The goal is not to memorize the training table; it is to work on NEW data. A model that nails the training set but fails on new inputs has OVERFIT - it learned noise and quirks instead of the underlying pattern. The standard guard is to hold out some labelled data as a VALIDATION set, never train on it, and watch the loss there. If training loss keeps dropping while validation loss rises, you are overfitting. Smaller models, more data, and regularization all help. The slip model is deliberately small (a few hundred parameters), which makes overfitting less likely.

2.7 Feature scaling (standardization)

Features often live on wildly different scales: a velocity might be 0.3 (m/s) while a joint speed is 4.0 (rad/s). Gradient descent works much better when all inputs are on a comparable scale. So we STANDARDIZE each feature: subtract its mean and divide by its standard deviation, computed over the training set, so every feature ends up roughly zero-mean and unit-variance:

```
x_scaled = (x - mean) / std
```

Crucially, the SAME mean and std must be applied at prediction time. In this project those numbers are saved inside the model file and applied automatically at inference, so deployment needs nothing extra (Section 6.1).

3. Neural network fundamentals

A neural network is a particular, very flexible family of parameterized functions. It is built by stacking simple units so the whole can represent complicated input-output relationships. You still train it with the loss + gradient descent recipe from Section 2.

3.1 The neuron: a weighted sum plus a bias

The basic unit takes several inputs, multiplies each by a weight, adds them up, adds a constant bias, and passes the result through an ACTIVATION function g :

```
inputs:  x1, x2, ..., xn
weights: w1, w2, ..., wn    bias: b

z = w1*x1 + w2*x2 + ... + wn*xn + b      (a weighted sum)
output = g(z)                          (activation applied)
```

The weights and bias are the parameters that learning adjusts. One neuron alone can only draw a straight-line (linear) boundary. The power comes from many neurons plus nonlinear activations.

3.2 Activation functions, and why nonlinearity matters

If you stack purely linear neurons, the whole stack collapses back into a single linear function - no matter how many layers. To represent curves and logical combinations ('slip only if joints fast AND body slow AND contact low'), you need a NONLINEAR activation between layers. Two we use:

```
ReLU (Rectified Linear Unit):  g(z) = max(0, z)
    cheap; passes positives, zeros out negatives. The standard
    hidden-layer activation - it lets the network bend.

Sigmoid:                      g(z) = 1 / (1 + e^(-z))
    squashes any real number into (0, 1). Used on the FINAL layer
    so the output reads as a probability - here, the slip score s.
```

So a hidden ReLU layer gives the network expressive power, and a final sigmoid turns the last number into a 0-to-1 confidence.

3.3 Layers and the multilayer perceptron (MLP)

Neurons are organized into LAYERS. All neurons in a layer read the same inputs (the previous layer's outputs) and produce a vector of outputs for the next layer. A stack of such fully-connected layers is a MULTILAYER PERCEPTRON (MLP). The middle layers are HIDDEN layers; their width (number of neurons) controls capacity. The slip model is an MLP with two hidden layers of 16 neurons each.

3.4 The forward pass in matrix form

Computing a layer's outputs for all its neurons at once is a matrix multiply. If a layer has 'in' inputs and 'out' neurons, its weights form an out-by-in matrix W and its biases a length-'out' vector b . The whole forward pass (input to output) is:

```
x0 = standardize(features)          # Section 2.7
x1 = ReLU( W1 @ x0 + b1 )            # hidden layer 1
x2 = ReLU( W2 @ x1 + b2 )            # hidden layer 2
s  = sigmoid( W3 @ x2 + b3 )         # output: slip score in (0,1)

(@ means matrix-vector multiply)
```

That is the entire run-time computation of a trained network: a few matrix multiplies and activations. It is cheap, which is why it can run inside the estimator at sensor rate. This same sequence is implemented three times in this project (PyTorch, NumPy, C++) and checked to agree (Section 6).

3.5 Backpropagation (how the gradients are found)

Training needs $d\text{Loss}/dw$ for every weight. BACKPROPAGATION is the algorithm that computes them: it runs the forward pass, then applies the calculus chain rule backward through the layers, reusing intermediate results so the cost is about the same as one forward pass. You do not implement it by hand - PyTorch records the operations and differentiates them automatically (Section 4.1). Conceptually: forward pass to get the loss, backward pass to get the gradients, then one gradient-descent step.

4. How PyTorch builds and trains the network

PyTorch is a Python library for building and training neural networks. The trainer in this repo (scripts/train_slip_model.py) uses it when available, and falls back to a plain NumPy linear model otherwise so the pipeline runs on machines without PyTorch.

4.1 Tensors and autograd

A TENSOR is PyTorch's array type (like a NumPy array, but it can also run on a GPU and, importantly, track operations for differentiation). When you compute with tensors that have `requires_grad` set, PyTorch builds a graph of the operations. Calling `loss.backward()` then walks that graph backward and fills in every parameter's gradient automatically. This is AUTOGRAD - it is why you never write backpropagation by hand.

4.2 Defining the model

`nn.Sequential` chains layers in order. `nn.Linear(in, out)` is a fully-connected layer (it holds the W matrix and b vector). `nn.ReLU` is the activation. The actual definition used here:

```
import torch.nn as nn
net = nn.Sequential(
    nn.Linear(8, 16), nn.ReLU(),      # 8 features  -> 16 hidden
    nn.Linear(16, 16), nn.ReLU(),    # 16          -> 16 hidden
    nn.Linear(16, 1),               # 16          -> 1 (a logit)
)
```

Note the final layer outputs ONE raw number with no sigmoid. That raw pre-sigmoid value is called a LOGIT. We handle the sigmoid in the loss (next) for numerical stability, and re-attach it at export time so the deployed model outputs a probability.

4.3 The loss: BCEWithLogitsLoss

`BCEWithLogitsLoss` combines a sigmoid and the binary cross-entropy of Section 2.4 into one numerically stable operation. It takes the raw logit and the true label and returns the BCE loss - without us ever explicitly computing sigmoid during training (which can overflow for large logits).

```
loss_fn = nn.BCEWithLogitsLoss()
```

4.4 The optimizer and the training loop

An OPTIMIZER applies the gradient-descent update of Section 2.5 to all parameters. Adam is a popular variant that adapts the step size per parameter and usually converges faster than plain gradient descent. The training loop is the recipe from Sections 2.5 and 3.5, written out:

```
opt = torch.optim.Adam(net.parameters(), lr=1e-2)
for epoch in range(epochs):
    opt.zero_grad()          # clear old gradients
    logits = net(X)          # forward pass on all examples
    loss = loss_fn(logits, y) # how wrong are we?
    loss.backward()          # autograd: fill in all gradients
    opt.step()               # one gradient-descent update
```

`zero_grad` clears gradients from the previous step (PyTorch accumulates them by default). `forward`, `backward`, `step` - that triad is the heart of training every neural network, no matter how large.

4.5 From a trained net to a portable file

After training, the knowledge lives in the layers' W matrices and b vectors. The exporter pulls those numbers out, tags the layers with their activations (`relu`, `relu`, `sigmoid` - re-attaching the sigmoid that training kept inside the loss), prepends the standardization mean/std, and writes a plain text file. From here on, PyTorch is no longer needed - the file fully describes the function (Section 6.1).

5. The slip model itself

5.1 What it predicts and why

The model maps the current locomotion situation to a slip score s in $[0,1]$. s near 0 means the feet are planted and leg odometry is trustworthy; s near 1 means slipping, so leg odometry should be distrusted. This single number is the bridge between machine learning and the Kalman filter.

5.2 The feature vector (the input x)

Eight numbers, assembled every leg-odometry update. The first three are DISAGREEMENT terms - the core signal, because under slip the commanded and joint motion continue while the measured body motion and contact collapse. The rest give context so the model can tell real slip from normal dynamics.

#	Feature	Why it signals slip
0,1	commanded minus measured body velocity (x, y)	Core signal: commanded motion persists, measured leg-odom velocity diverges.
2	commanded minus measured yaw rate	Same idea for turning: command vs IMU-measured rotation.
3	leg-odometry speed	Context: slip looks different at low vs high speed.
4,5	joint velocity mean and max	Legs keep cycling under slip even when the body is not moving.
6	horizontal motion-acceleration magnitude	Slip events cause horizontal jerks the IMU sees.
7	fraction of feet in contact	Slip correlates with reduced or irregular foot contact.

The layout is defined once (SlipFeatures in slip_model.hpp) and mirrored in Python, so the feature order can never drift between training and inference.

5.3 The architecture

A standardization step followed by a 2-hidden-layer MLP, exactly the forward pass of Section 3.4:

```
features (8)
  -> standardize: (x - mean) / std          [stored in the model file]
  -> Linear(8 -> 16) -> ReLU
  -> Linear(16 -> 16) -> ReLU
  -> Linear(16 -> 1) -> Sigmoid
  -> slip score s in (0,1)
```

Why an MLP and not a hand-written threshold? Because slip is a nonlinear combination of conditions. 'Joints fast' is only suspicious when ALSO 'body slow' and 'contact low'. A single threshold cannot express that conjunction; a small MLP learns it from data.

5.4 Turning the score into filter behaviour

The estimator's leg-odometry update has a measurement-noise covariance R_{leg} . Larger R means 'this measurement is noisier', so the filter moves its estimate less in response to it. We inflate R using the slip score:

```
R_leg <- R_base * (1 + lambda * s)^2
```

At $s=0$ the factor is 1 (leg odometry fully trusted). The square is there because R is a variance: scaling the standard deviation by $(1 + \lambda s)$ squares into the variance. λ is a sensitivity knob - larger λ makes the filter abandon leg odometry faster as slip rises. The result is a smooth, monotonic dial rather than a hard on/off switch.

5.5 Where the training data comes from

Ideally: real logs from sim or hardware runs, labelled as slip / no-slip (pass them with `--data`). Until those exist, the trainer uses a physics-inspired SYNTHETIC dataset. It samples a hidden slip state and then generates feature values consistent with it: under slip, measured body velocity collapses relative to commanded, contact drops, and acceleration spikes. This gives the network genuine structure to learn, so the whole pipeline is runnable before real data is collected. The synthetic data is a scaffold, not the final word - on-robot quality requires real labelled logs.

6. Deployment: train in Python, run in C++

A robot estimator should not depend on a heavy machine-learning runtime. So training happens offline in Python/PyTorch, and the trained model is exported to a tiny text file that a few lines of C++ can evaluate with no ML library at all - just linear algebra (Eigen).

6.1 The weights file: the contract

A plain-text format lists the input dimension, the standardization mean and std, then each layer's shape, activation, weight matrix, and bias vector. It is framework-agnostic: both the PyTorch trainer and the NumPy fallback write it, and both the C++ and NumPy inference paths read it.

```
input_dim 8
mean  <8 numbers>
std   <8 numbers>
layers 3
layer 8 16 relu
<16 rows of weights, 8 numbers each> <then 16 biases>
layer 16 16 relu
...
layer 16 1 sigmoid
...
```

Choosing plain text over a format like ONNX or TorchScript is deliberate: loading it needs only string-to-number parsing, and evaluating it needs only matrix multiplies. No libtorch to cross-compile for the robot.

6.2 Inference in C++ with Eigen

SlipModel (include/go2_eskf/slip_model.hpp) is header-only and depends only on Eigen. It parses the file into matrices and runs the forward pass of Section 3.4. The output feeds the covariance rule of Section 5.4. This is what actually runs on the robot, inside the estimator node.

6.3 The NumPy twin and cross-validation

A line-for-line NumPy copy of the forward pass (scripts/slip_reference.py) exists alongside the C++. A cross-validation script feeds the same inputs to both and checks they produce the same outputs - they agree to about $1e-15$ (machine precision). This is the project's standard discipline: two independent implementations that must match turn the math into its own regression test. Any bug on either side breaks the match immediately.

6.4 Integration into the estimator node

When enabled, the node subscribes to the commanded velocity and joint states, gathers the IMU-derived features during prediction, assembles the 8-feature vector on each leg-odometry update, runs SlipModel, inflates R_{leg} , and publishes the slip score for inspection. If the model file is missing or fails to load, the node logs a warning and falls back to a fixed covariance - slip adaptivity can never crash localization.

7. Reproduce the whole pipeline

```
# 1. Train (PyTorch if installed, else NumPy fallback) -> weights file
python3 src/go2_eskf/scripts/train_slip_model.py \
    --out src/go2_eskf/config/slip_model.txt

# 2. Confirm C++ and NumPy forward passes agree (~1e-15)
python3 src/go2_eskf/scripts/cross_validate_slip.py

# 3. Run the estimator with the slip model enabled
ros2 run go2_eskf go2_eskf_node --ros-args -p use_slip_model:=true \
    -p slip_model_path:=<ws>/install/go2_eskf/share/go2_eskf/config/slip_model.txt

# 4. Retrain on REAL labelled logs once you have them
python3 src/go2_eskf/scripts/train_slip_model.py --data runs.csv \
    --out src/go2_eskf/config/slip_model.txt
```

The `--data` CSV has the 8 feature columns followed by a final 'slip' label column in [0,1]. Everything downstream (export, cross-validation, C++ inference) is identical whether the model was trained on real or synthetic data.

8. Limitations and natural extensions

- The shipped model is trained on synthetic data. The architecture and pipeline are proven; on-robot accuracy needs real labelled logs (`--data`).
- The contact-fraction feature is constant (1.0) until a foot-contact source is wired into the node.
- The covariance inflation is isotropic - it scales the x and y leg-velocity noise equally. Directional slip (e.g. only sideways) is not distinguished; a 2x2 extension could.
- The score is computed per update with no memory. A short temporal smoothing or hysteresis on s would reduce flicker between slip and no-slip.
- No validation split is used during the synthetic-data demo. Add one when training on real data to watch for overfitting (Section 2.6).

Appendix - Glossary

Term	Meaning
Feature	One input number describing the situation; the model reads a fixed-length feature vector.
Label	The correct answer for a training example (here 1=slip, 0=no slip).
Parameter / weight	An internal number the training algorithm adjusts to fit the data.
Model	A parameterized function $y = f(x; \theta)$ mapping features to a prediction.
Loss	A single number measuring how wrong the model is; training minimizes it.
BCE	Binary cross-entropy, the loss used for two-class probability outputs.
Gradient descent	Iteratively nudging parameters opposite the loss gradient to reduce loss.
Learning rate	The step size of each gradient-descent update.
Epoch	One full pass over the training dataset.
Overfitting	Fitting training noise so the model fails on new data.
Standardization	Rescaling each feature to zero mean and unit variance.
Neuron	A unit computing a weighted sum of inputs plus bias, then an activation.
Activation	A nonlinear function (ReLU, sigmoid) applied to a neuron's weighted sum.
ReLU	$\max(0, z)$; the standard hidden-layer activation.
Sigmoid	$1/(1+e^{-z})$; squashes a number into (0,1), used for probabilities.
Logit	A raw pre-sigmoid score; $\text{sigmoid}(\text{logit})$ is a probability.
MLP	Multilayer perceptron: a stack of fully-connected layers with activations.
Forward pass	Computing the output from the input through all layers.
Backpropagation	Chain-rule algorithm that computes the loss gradient for every weight.
Autograd	PyTorch feature that records operations and differentiates them automatically.
Tensor	PyTorch's array type (supports GPUs and gradient tracking).
Optimizer	Object (e.g. Adam) that applies gradient-descent updates to parameters.
Covariance (R)	In a Kalman filter, how noisy a measurement is; larger R = trusted less.